PANDUAN BUILD WORKSHOP

Bangun AI Project Manager Anda

Referensi bawa-pulang: konsep, tiap langkah kode, dan cara menjalankannya — CrewAI + Ollama lokal.

Ini pendamping sesi praktik dan referensi Anda setelahnya. Berkas yang bisa dijalankan (pm_crew.py, tool_demo.py, virtual_office.py) ada di bundel kode, dan tiap tahap build punya berkas siap jalan di checkpoints/ jika Anda tertinggal; panduan ini menjelaskan apa yang dilakukan tiap langkah dan mengapa.

Model mental

- **Agent —** sebuah LLM plus tools, memory, dan sebuah loop. Tanpa hype, hanya itu.
- **Loop ReAct —** tiap putaran agent berpikir (Thought), bertindak/Action (memanggil tool), mengamati/Observe hasilnya, dan mengulang hingga tujuan tercapai. CrewAI menjalankan loop ini untuk Anda.
- **Crew —** satu agent untuk tugas terfokus; satu crew spesialis untuk pekerjaan kompleks. Sequential = pipeline; seorang Lead juga bisa mendelegasikan ke spesialis.
- **Stack lokal —** CrewAI mengorkestrasi, terhubung langsung ke Ollama, Ollama menjalankan open weights (Llama 3.1 / Llama 3.2) di mesin Anda. Tanpa API key, tanpa data keluar dari laptop.

Setup

Instal model lokal sekali, buat environment Python bersih, lalu tambahkan framework-nya.

```
# 1. install Ollama (ollama.com), then pull open models
ollama pull llama3.1          # ~4.7 GB, runs fully local
ollama pull llama3.2:3b       # ~2 GB, for weaker laptops

# 2. a clean Python 3.10-3.13 environment + the framework
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# 3. sanity check: is everything ready?
python check_setup.py
```

Membangun asisten PM, langkah demi langkah

1. Agent pertama Anda

Sebuah Agent, sebuah Task, dan sebuah Crew untuk menjalankannya — diarahkan ke model Ollama lokal.

```
from crewai import Agent, Task, Crew, Process, LLM

# point CrewAI at the LOCAL model served by Ollama
llm = LLM(model="ollama/llama3.1",
          base_url="http://localhost:11434")
```

```
planner = Agent(
    role="Project Planner",
    goal="Turn a project brief into a clear task list",
    backstory="A sharp PM who makes vague briefs actionable.",
    llm=llm, verbose=True,
)

task = Task(
    description="Break down this brief into tasks:\n{brief}",
    expected_output="A numbered task list with an owner each",
    agent=planner,
)

crew = Crew(agents=[planner], tasks=[task], process=Process.sequential)
print(crew.kickoff(inputs={"brief": open("brief.md").read()}))
```

Mengapa: Satu agent + satu task + satu crew adalah asisten berfungsi dalam ~20 baris.

2. Beri sebuah tool

Tool hanyalah fungsi Python. Jika model memutuskan membutuhkannya, CrewAI memanggilnya dan mengembalikan hasilnya.

```
from crewai.tools import tool
import json

@tool("save_tasks", max_usage_count=1)
def save_tasks(tasks_json: str) -> str:
    "Save a JSON list of task titles to tasks.json."
    data = json.loads(tasks_json)
    with open("tasks.json", "w") as f:
        json.dump(data, f, indent=2)
    return f"saved {len(data)} tasks"

# the agent may now decide to call this tool
planner.tools = [save_tasks]
```

Mengapa: Model yang memutuskan apakah memanggil tool, jadi tool bisa terlewat. Jalankan tool_demo.py beberapa kali untuk melihatnya. Ekspos hanya tool yang benar-benar diperlukan agent (least privilege).

3. Output bertipe, bukan prosa

Buat planner mengembalikan objek Pydantic tervalidasi agar langkah berikutnya bisa memercayai datanya.

```
from pydantic import BaseModel
from typing import List

class TaskItem(BaseModel):
    title: str
    owner: str
    estimate_days: int

class Plan(BaseModel):
    tasks: List[TaskItem]

plan_task = Task(
    description="Break the brief into tasks:\n{brief}",
    expected_output="A Plan object",
    agent=planner,
    output_pydantic=Plan,     # validated, typed output
)
```

Mengapa: Output bertipe membuat agent bisa dirangkai — tanpa regex pada prosa.

4. Tambah dua spesialis

Seorang risk analyst dan seorang status writer, masing-masing satu tanggung jawab sempit.

```
risk = Agent(
    role="Risk Analyst",
    goal="Flag the top risks, with one mitigation each",
    backstory="A cautious PM who has watched projects slip.",
    llm=llm)

writer = Agent(
    role="Status Writer",
    goal="Draft a short status update for stakeholders",
    backstory="Writes clear, calm updates and never invents facts.",
    llm=llm)

risk_task = Task(description="Review the plan. List the top risks.",
                 expected_output="A risk register", agent=risk,
                 context=[plan_task])

status_task = Task(description="Write a kickoff update. Work has NOT started: claim no
progress.",
                   expected_output="At most 5 lines", agent=writer,
                   context=[plan_task, risk_task])
```

Mengapa: Operkan konteks antar-task dengan context=[...] agar tiap agent melihat output sebelumnya.

5. Jalankan PM crew

Rangkai tiga agent menjadi satu sequential crew lalu jalankan pada brief nyata.

```
from pathlib import Path

crew = Crew(
    agents=[planner, risk, writer],
    tasks=[plan_task, risk_task, status_task],
    process=Process.sequential,     # planner -> risk -> writer
    verbose=True,
)

result = crew.kickoff(inputs={"brief": open("brief.md").read()})
print(result)      # the final task's output: the status update

# save shared state from Python, so it can't depend on the model's choice
plan = result.tasks_output[0].pydantic
Path("project_state.json").write_text(plan.model_dump_json(indent=2))
```

Mengapa: Itulah AI project manager Anda — sekitar 40 baris. print(result) hanya menampilkan output task terakhir; result.tasks_output menyimpan output semua task.

Intermezzo: kantor virtual (NexaPaw)

Beralih dari pipeline tetap ke seorang Lead bernama yang mengembalikan rencana kerja bertipe, yang disalurkan Python ke tim spesialis — pola di balik kantor NexaPaw milik Felix, di mana tiap Claude Code role adalah agent (CEO dan CTO di atas para project lead di atas para engineer spesialis), divisualisasikan live di HQ 3D. Anda definisikan staf; Lead menentukan siapa mengerjakan apa, dan kapan.

```
def staff(role, goal, backstory):
    return Agent(role=role, goal=goal, backstory=backstory, llm=llm, verbose=True)

lead = staff("Project Lead", "Assign work to the right specialist", "Runs a NexaPaw project
team.")
team = {
    "CV Engineer": staff("CV Engineer", "Build and evaluate the model", "A pragmatic CV
engineer."),
    "Backend Engineer": staff("Backend Engineer", "Build the APIs and data pipeline", "A
careful backend engineer."),
    "Frontend Engineer": staff("Frontend Engineer", "Build the user interface", "A tidy
frontend engineer."),
    "QA Engineer": staff("QA Engineer", "Test the work against the spec", "A sharp, fair
tester."),
}

class Assignment(BaseModel):     # a role must name a real staff member
    role: Literal["CV Engineer", "Backend Engineer", "Frontend Engineer", "QA Engineer"]
    work: str

class WorkPlan(BaseModel):
    assignments: List[Assignment]

# 1. the Lead decides who does what
planning = Task(description=f"Assign this goal to your team ({', '.join(team)}): {brief}",
                expected_output="A WorkPlan with at most 3 assignments", agent=lead,
                output_pydantic=WorkPlan)
plan = Crew(agents=[lead], tasks=[planning]).kickoff().pydantic

# 2. Python dispatches the Lead's decisions to the specialists
tasks = [Task(description=f"{a.work}. Use at most 4 bullet points.", agent=team[a.role],
              expected_output="At most 4 bullet points") for a in plan.assignments]
result = Crew(agents=list(team.values()), tasks=tasks).kickoff()
print(result)
```

TRADE-OFF JUJUR

Kantor virtual itu kuat dan lebih sulit dikendalikan. Dalam pengujian pada Llama 3.1 dan 3.2, Process.hierarchical tak pernah sampai ke spesialis: selesai tanpa error dengan delegasi tertulis sebagai teks, sehingga build ini memakai delegasi bertipe. Selalu periksa siapa yang benar-benar berjalan. Lebih banyak agent = lebih banyak panggilan model = lebih lama dan nondeterministik. Pakai hanya saat satu agent mentok.

Membuatnya nyata

Validasi output, batasi iterasi, dan tahan tindakan berdampak di belakang manusia.

```
# guardrail: Python validates the output; failures go back to the agent
def short_update(output: TaskOutput) -> tuple[bool, Any]:
    if len(output.raw.split()) > 120:
        return (False, "Too long. Keep it under 120 words.")
    return (True, output.raw)

status_task = Task(description="...", expected_output="...", agent=writer,
                   guardrail=short_update, guardrail_max_retries=2)

# gate risky actions behind a human — draft, never send
@tool("send_email")
def send_email(to: str, body: str) -> str:
```

```
    "Save a draft for human approval; does NOT send."
    Path("outbox.md").write_text(body)
    return "DRAFT saved to outbox.md — needs approval"

# stop a stuck agent from looping forever
planner = Agent(role="Project Planner", goal="...", backstory="...",
                llm=llm, max_iter=8, max_retry_limit=2)
```

- Aktifkan verbose / logging untuk melihat tiap thought, panggilan tool, dan hasil.
- Lacak token dan latensi — model lokal gratis tetapi lebih lambat; cache panggilan berulang.
- Evaluasi pada set brief tetap dengan output yang diketahui benar sebelum dipercaya.
- Local-first menjaga data pelanggan dari server pihak ketiga (membantu UU PDP).
- Waspadai prompt injection saat agent membaca teks eksternal, email, atau halaman web.

ATURAN SEDERHANA

Agent sebaiknya mengusulkan; manusia yang memutuskan — untuk hal apa pun yang penting (kirim, bayar, hapus, atau keputusan tentang orang).

Jalankan kode jadinya

- ollama pull llama3.1 (sekali)
- pip install -r requirements.txt
- python pm_crew.py --brief brief.md — merencanakan tugas, menandai risiko, menyusun status update
- python tool_demo.py — satu agent memilih apakah memanggil tool
- python virtual_office.py — Lead merencanakan, lalu para spesialis berjalan. Laptop lemah? WORKSHOP_MODEL=ollama/llama3.2:3b python virtual_office.py

---

Disusun oleh Felix Yustian Setiono untuk MurzEdu Program (ecommurz). Kode adalah titik awal — sesuaikan dengan stack Anda.